Project Report On

Predictive Weather Guidance With Route Engine

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

NIMISH M BIJU

41523404089

(SIXTH SEMESTER)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEETING
UNIVERSITY COLLEGE OF ENGINEERING, KARIAVATTOM
THIRUVANANTHAPURAM APRIL 2026

CERTIFICATE

This is to certify that the seminar report entitled "**Predictive Weather Guidance With Route Engine**"

Is a Bonafide record of the Seminar presented in the sixth semester by

NIMISH M BIJU

41523404089

In partial fulfillment for the award of Degree of Bachelor of Technology in

Computer Science and Engineering

From the university of kerala for the academic year 2023-2027

Project Supervisor | Project Coordinator | Head of Department

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to Prof. Saina A R, Principal, University College of Engineering, Kariyavattom , for the facilities she has provided in the college without which the project could not have been completed.

I acknowledge my sincere gratitude to Ms. Divya D.T., Head of the Department, Department of Computer Science and Engineering, University College of Engineering, Kariyavattom, for her support and Encouragement.

I am greatly indebted to my Seminar Guide Ms. Gargy G, Assistant Professor, Department of Computer Science and Engineering, University College of Engineering, Kariyavattom for her valuable guidance and Encouragement.

I also acknowledge my gratitude to all other members of all faculty of the Department of Computer Science and Engineering.

We also express our wholehearted gratefulness to the College Management and to all those who helped me make this undertaking a success.

Above all, we thank our Parents and God Almighty without whose blessings this seminar would not have been a reality.

Last but not least, I am also thankful to all my Friends who gave me valuable suggestions and helped me completion of this seminar work.

Thank you

ABSTRACT

Traditional navigation systems calculate the fastest route between two points but frequently fail to account for dynamic environmental factors, such as changing weather conditions over the duration of a journey. This project proposes and implements a **Predictive Weather Guidance and Route Planning System**, a web-based application designed to integrate meteorological forecasting directly into the pathfinding process. Built entirely as a client-side application, the system utilizes an asynchronous architecture combining the OpenStreetMap Nominatim API for geocoding, Leaflet Routing Machine (OSRM) for trajectory mapping and travel time estimation, and the OpenWeather API for fetching temporal meteorological data. Unlike standard applications that display static current weather, this system mathematically predicts the user's exact arrival time at critical waypoints (Start, Midpoint, and Destination) based on user-defined departure times and driving distance. It then queries a 5-day predictive weather model to extract the specific 3-hour forecast block corresponding to those future arrival times. Furthermore, the system includes an automated hazard-detection algorithm that analyzes the fetched weather codes to generate real-time safety alerts for rain, strong winds, fog, and thunderstorms. By successfully bridging spatial navigation with predictive meteorology in a seamless "glassmorphism" user interface, this project demonstrates a highly practical approach to Advanced Traveler Information Systems (ATIS), ultimately enhancing driver awareness, route planning efficiency, and overall road safety.

TABLE OF CONTENTS

TABLE OF CONTENTS

ABSTRACT 4

CHAPTER 1 INTRODUCTION 9

1.1 Background 9

1.2 Problem Statement 10

1.3 Objectives 11

1.4 Scope and Significance 12

CHAPTER 2 LITERATURE REVIEW AND MOTIVATION 13

2.1 Weather as a Mobility Variable 13

2.2 Shortcomings of Conventional Navigation Systems 14

2.3 API Integration as a Decision-Support Architecture 15

2.4 Algorithmic Perspectives from the Literature 16

2.5 Static Versus Dynamic Route Guidance 17

2.6 User Interface and Decision-Support Lessons 18

2.7 Research Gap and Relevance 19

CHAPTER 3 SYSTEM ANALYSIS AND PROPOSED DESIGN 20

3.1 Existing Fragmented Workflow 20

3.2 Proposed System 21

3.3 Overall Architecture 22

3.4 Functional and Non-Functional Requirements 23

3.5 Feasibility Analysis 24

CHAPTER 4 ALGORITHMS AND TECHNICAL WORKFLOW 25

4.1 Technical Workflow Overview 25

4.2 Detailed Algorithm Descriptions 26

CHAPTER 5 DATASET AND TECHNOLOGY STACK 27

5.1 Data Sources Used by the System 27

5.2 Technology Stack 28

CHAPTER 6 MODULE DESCRIPTION AND IMPLEMENTATION DETAILS ... 28

6.1 User Interface Module 28

6.2 Current Location and Input Validation 29

6.3 Route Summary Module 29

6.4 Weather Card Module 30

6.5 Warning Module 30

6.6 Event-Driven Control Flow 31

CHAPTER 7 OUTPUT, DISCUSSION, ADVANTAGES AND LIMITATIONS ... 31

7.1 Output and Discussion 31

7.2 Advantages 31

7.3 Limitations 31

CHAPTER 8 CONCLUSION AND FUTURE SCOPE 31

8.1 Conclusion 31

8.2 Future Scope 31

CHAPTER 9 REFERENCES 31

CHAPTER 10 PUBLICATIONS 32

LIST OF TABLES

Table 1: Comparison of Existing Fragmented Workflow and Proposed System 21

Table 2: External Services Used by the Application 27

Table 3: Route-Point and Forecast-Time Matching Concept 26

Table 4: Technical Stack and Implementation Role 28

CHAPTER 1
INTRODUCTION

1.1 Background

Digital navigation has become a routine component of commuting, tourism, logistics, and inter-city travel. In its simplest form, a route planner receives an origin and a destination and returns a path through the road network, normally optimized for shortest distance or minimum estimated travel time. Although this form of optimization is useful, it does not fully represent the realities of travel. Real-world journeys are affected not only by road geometry and distance, but also by weather, visibility, atmospheric hazards, and the expected time at which each route segment is reached.

Conventional travel planning is therefore fragmented. Users often consult one system for navigation and another system for weather. A route application tells them how long the trip may take, while a weather application tells them what the forecast looks like for one or more places. The user must mentally combine these outputs and infer whether the route is still suitable under forecasted conditions. That manual interpretation becomes difficult when the departure is planned for later in the day, when forecast panels are organized into discrete time blocks, or when different parts of the route traverse different localities.

The present project, Predictive Weather Guidance with Route Planning, addresses this gap by coupling route geometry with route timing and then coupling that timing with forecast data. The result is not merely a route planner with a weather panel and not merely a weather app with a map. It is a route-aware forecasting interface that attempts to estimate what weather will be encountered when the traveller reaches each important point of the journey.

1.2 Problem Statement

- Existing route planning systems usually optimise for distance or travel time, but they do not explicitly align forecasted weather with the user's planned travel timeline.
- Users typically need to open multiple applications and manually compare route duration with forecast panels, which increases cognitive burden and may lead to poor judgement.

- Weather conditions vary along a route; conditions at the origin, intermediate segment, and destination may differ considerably, especially for longer or geographically diverse trips.
- A browser-based weather-aware route planner is needed to provide route-specific forecast guidance and warning messages in a single interface using live geospatial and forecast services.

1.3 Objectives

- To design a single-page web application that accepts a source, destination, and departure time from the user.
- To geocode human-readable place names into latitude–longitude coordinates and support reverse geocoding for route-derived points.
- To compute a drivable road route, display it on an interactive map, and extract route summary information such as total distance and estimated travel duration.
- To determine key route points, specifically the route start, the midpoint derived from the route polyline, and the destination.
- To calculate estimated arrival times for those points by combining departure time with route duration.
- To fetch weather forecast data for each route point and match the route timeline to the closest forecast timestamps.
- To generate contextual weather-risk warnings for conditions such as rain, thunderstorm, low visibility, and strong wind.
- To implement the full system in a frontend-only architecture suitable for educational deployment and mini-project demonstration.

1.4 Scope and Significance

The scope of the present project is deliberately focused. It does not attempt to build a commercial-grade navigation platform, nor does it claim to perform full route optimization under continuously changing weather fields. Instead, it demonstrates a clear and educationally valuable pipeline: place names are converted to coordinates, coordinates are converted to a route, the route is interpreted in time, and time-indexed forecast records are matched to route points.

The significance of the project lies in decision support. A traveller planning a trip from Kochi to Thrissur, for example, may want to know whether rain is likely during departure, whether the midpoint region may face thunderstorms, or whether strong wind is expected near the destination. The application automates that reasoning. This makes the system more informative than a destination-only weather check and more context-aware than a route-only map view.

CHAPTER 2

LITERATURE REVIEW AND MOTIVATION

2.1 Weather as a Mobility Variable

The paper by Ye, Li, Das, and Du (2023) provides a strong conceptual foundation for this project. In the abstract and introduction, the authors argue that a notable shortcoming in contemporary digital navigation systems is their failure to incorporate live weather data. They emphasize that weather significantly affects transportation across air, sea, and land, and that adverse conditions such as heavy rain, strong winds, and dense fog can cause delays, safety risks, and logistical disturbances. The paper also notes that heatwaves and extreme temperatures affect non-motorized travelers such as pedestrians and cyclists, demonstrating that weather is not a marginal variable but a core mobility factor.

The relevance of this observation to the present application is direct. Predictive Weather Guidance with Route Planning is built on the same assumption: weather should be treated as part of route interpretation rather than as a disconnected external layer. Although our application is smaller in scale and implemented differently, the paper validates the underlying design principle that route guidance becomes more useful when meteorological conditions are considered in relation to travel.

2.2 Shortcomings of Conventional Navigation Platforms

Ye et al. further explain that mainstream navigation tools usually anchor their recommendations on metrics such as distance and projected travel time. These metrics are essential, but they are incomplete because they fail to account for rapidly changing atmospheric conditions. The paper notes that such omissions can range in consequence from minor

inconvenience to serious safety implications, because users may be guided into areas affected by hazardous weather or unexpected delays. This criticism is highly relevant to the current web app because the project is explicitly designed to address that omission at a mini-project level.

The paper therefore helps strengthen the problem statement of the present work. Our application does not challenge the importance of distance and time; rather, it extends them. The route still comes first, but the route becomes much more meaningful when the estimated time of arrival at route points is used to retrieve the forecast expected at those times.

2.3 API Integration as a Decision-Support Architecture

In the methodology section, Ye et al. describe a navigation tool that combines Google Directions API with OpenWeather API. Their stated goal is to provide users not just with navigation but with informative guidance that enables them to choose routes while avoiding unfavorable weather conditions. The paper explains that route details and weather information are separate data streams that must be harmonized at the application layer. This idea is highly relevant to the present project, even though our implementation uses a different stack: Nominatim, Leaflet Routing Machine over OSRM, and OpenWeather.

The architecture reported in the paper confirms a general pattern for weather-aware route systems: one service resolves route structure and travel timing, another service provides environmental information, and the application layer integrates them into user-facing guidance. Our application follows the same high-level logic, but uses a fully frontend browser workflow and replaces the Google-based route service with open geospatial services. The result is lighter and easier to deploy, though naturally less feature-rich than a commercial multi-service stack.

2.4 Algorithmic Perspectives from the Literature

The paper also enriches the theoretical background by discussing routing algorithms such as Dijkstra's algorithm and A* algorithm. The authors explain that Dijkstra's algorithm is central to shortest-path computation and that modern route-planning research often modifies such algorithms to include safety-related factors rather than optimizing only for length. They additionally describe A* as a heuristic path-planning algorithm that evaluates candidate paths using both path cost and heuristic distance to the goal.

It is important not to overstate the relationship between this literature and the present application. The uploaded script does not implement Dijkstra or A* directly in JavaScript. Instead, the application delegates route finding to Leaflet Routing Machine and the underlying routing service. Nevertheless, the literature remains relevant because it explains the algorithmic class of problems that route engines solve and shows that routing can be extended beyond pure shortest-distance logic toward safety-aware decision support.

2.5 Static Versus Dynamic Route Guidance

Another important contribution of the paper is its distinction between static and dynamic routing algorithms. Static approaches rely on fixed or historical criteria, whereas dynamic approaches adjust to real-time information such as traffic or changing conditions. Ye et al. emphasize that dynamic algorithms are particularly valuable because they adapt routes based on live contextual data. This discussion helps frame the current project as a partially dynamic system: it is not dynamic in the sense of continuously recomputing the route as weather changes, but it is dynamic in the sense that route guidance is generated from live geocoding, live route computation, and live forecast retrieval at the time the user interacts with the app.

This distinction also reveals the limits of the present design. Our application retrieves forecasts and then interprets them at selected route points; it does not yet re-route dynamically based on changing forecast severity or continuously update the user's path while moving. The literature therefore provides a useful benchmark against which the current system can be described honestly: it is an intelligent forecast-alignment tool rather than a fully adaptive route-optimization platform.

2.6 User Interface and Decision-Support Lessons from the Paper

Ye et al. present a weather-aware navigation tool interface with multiple coordinated components, including inputs, weather-mode controls, a date selector, route retrieval, a line graph of temperature fluctuation along routes, and a main map panel. Their discussion is valuable because it shows that weather-aware navigation is not only a computational problem but also a user-interface problem. Data must be presented in a way that helps users make decisions rather than simply exposing raw forecast numbers.

The present web application preserves that same decision-support emphasis, but adapts it to a simpler and more focused interface. Instead of a multi-mode dashboard and route temperature graph, our app uses a glass-panel overlay with focused inputs, a route summary card, three weather cards, and automatic warning banners. This design choice is consistent with the educational purpose of the project: it reduces cognitive load and keeps the interaction centered on a single route and three key route points.

2.7 Research Gap and Relevance to the Present Web Application

Taken together, the paper strengthens the literature review in two ways. First, it establishes that integrating route information with weather information is a recognized and worthwhile research problem. Second, it shows that such systems can meaningfully improve travel decision support by exposing weather conditions in route context. However, the paper's reported tool focuses strongly on temperature-oriented route interpretation and a Google-centered API architecture, while our application focuses on forecast matching for the start, midpoint, and destination in a browser-only environment using open services.

This difference helps clarify the identity of the current web application. The essence of Predictive Weather Guidance with Route Planning should not be lost while incorporating the literature. The app we created is not a replica of the paper's tool. It is a frontend route-weather guidance system that preserves the same conceptual motivation — weather-aware travel support — while implementing it through Kerala-centered route demonstrations, three-point forecast matching, and automatically generated route-specific weather warnings.

CHAPTER 3

SYSTEM ANALYSIS AND PROPOSED DESIGN

3.1 Existing Fragmented Workflow

In the conventional fragmented workflow, a user computes a route in one application and checks weather in another. This means that no system automatically explains what the weather will be when the user reaches a route segment. Instead, the user must infer that manually. Such inference can be especially unreliable when the departure is planned for later in the day, when forecast panels are organized into discrete time blocks, or when different parts of the route traverse different localities.

From a systems-design perspective, the fragmented approach fails at integration rather than at data availability. Route data exist, and weather data exist, but they are not synchronized. The present application addresses exactly that integration gap.

3.2 Proposed System

The proposed system integrates user input capture, autocomplete, geocoding, route generation, midpoint extraction, reverse geocoding, timeline generation, forecast retrieval, nearest-time forecast matching, warning generation, and result rendering in a single-page browser application. The user provides the route and departure information once, and the application completes the remaining workflow automatically.

The HTML, CSS, and JavaScript confirm this design. The HTML declares the structural placeholders for map, inputs, summary, warnings, and weather cards. The CSS defines the visual and responsive behavior. The JavaScript performs the operational orchestration that binds all system components together.

Aspect	Existing fragmented workflow	Proposed application
User interaction	Route and weather checked separately	Single integrated planning workflow
Time matching	Estimated manually by the user	Computed automatically from route duration
Intermediate weather insight	Usually absent	Midpoint derived and analysed
Safety interpretation	Left to user judgement	Automatic warning banners generated
Hosting model	Varies by service	Static frontend deployment possible

Table 1. Comparison of existing fragmented workflow and proposed system

3.3 Overall Architecture

The application can be understood as a layered architecture moving from user input, through geocoding and routing, into route processing and timeline generation, and finally into forecast retrieval, warning generation, and presentation. This layered view is useful because it separates concerns: location interpretation, route geometry, temporal reasoning, environmental interpretation, and user communication are different technical problems even though they are executed in one browser workflow.

3.4 Functional and Non-Functional Requirements

Functional requirements include route generation, forecast retrieval, time-based forecast matching, and warning generation. Non-functional requirements include responsiveness, clarity of interaction, reasonable browser-side performance, lightweight deployment, and a visually

coherent interface. The project overview also emphasizes cost effectiveness, privacy-respecting behavior, and ease of hosting because no persistent backend is used.

Functional Requirements

- To design a single-page web application that accepts a source, destination, and departure time from the user.
- To geocode human-readable place names into latitude–longitude coordinates and support reverse geocoding for route-derived points.
- To compute a drivable road route, display it on an interactive map, and extract route summary information such as total distance and estimated travel duration.
- To determine key route points, specifically the route start, the midpoint derived from the route polyline, and the destination.
- To calculate estimated arrival times for those points by combining departure time with route duration.
- To fetch weather forecast data for each route point and match the route timeline to the closest forecast timestamps.
- To generate contextual weather-risk warnings for conditions such as rain, thunderstorm, low visibility, and strong wind.
- To implement the full system in a frontend-only architecture suitable for educational deployment and mini-project demonstration.

Non-Functional Requirements

- The interface should remain responsive on desktop and mobile screens.
- The application should remain understandable without page reloads or multiple windows.
- The browser should provide clear feedback during route calculation and weather retrieval.
- The design should remain lightweight and easy to host because there is no backend.

3.5 Feasibility Analysis

Technical feasibility is high because all required tasks are available through documented APIs and JavaScript libraries. Browser-side asynchronous requests are sufficient for autocomplete, geocoding, forecast retrieval, and user-interface updates.

Operational feasibility is also high because the interaction model is simple. The user enters route information and departure time, then presses a single planning button. Results are progressively displayed without page reload.

Economic feasibility is favorable because the design relies on open or free-tier services and does not require a dedicated application server or database for prototype use.

CHAPTER 4

ALGORITHMS AND TECHNICAL WORKFLOW

4.1 Technical Workflow Overview

The application is driven by a chain of dependent algorithms rather than by a single monolithic routine. Each stage produces output that becomes the input for the next stage. The geocoding stage produces coordinates, the routing stage produces a polyline and duration, the timeline stage converts duration into expected arrival times, the forecast stage returns time-indexed weather records, and the matching stage selects the most relevant records for presentation. This modular structure makes the application easier to reason about and is consistent with the function-oriented design visible in the uploaded JavaScript file.

The first major algorithm is the road mapping and path computation algorithm. It begins by reading the source and destination text entered by the user. If those fields already contain coordinates stored from an autocomplete selection, the application reuses them. Otherwise, it calls a fallback geocoding routine. Once coordinates are available, the application instantiates a Leaflet Routing Machine control using the two route endpoints. When routing completes, the system reads the route summary and the ordered coordinate array returned in the route object. The midpoint is derived as the coordinate located at the mathematical center of that array. This is a route-based midpoint and therefore differs from a straight-line geographical midpoint between source and destination.

The second major algorithm is weather forecast extraction and API integration. The project overview states that OpenWeather 5-day forecast data are retrieved in 3-hour chunks. The code confirms this by calling the OpenWeather forecast endpoint rather than a current-weather endpoint. For each selected route point—start, midpoint, and destination—the application builds a request URL using latitude, longitude, the API key, and metric units. Instead of waiting for each forecast sequentially, the function `triggerWeatherPrediction()` launches all three requests

concurrently using `Promise.all`. This reduces the total waiting time perceived by the user and is a good example of asynchronous orchestration in browser JavaScript.

Route point	Target time	Computation rule	Forecast selection
Start	Departure time	t0 = departure time	Nearest forecast to t0
Midpoint	Half of route duration	t1 = departure + totalDuration/2	Nearest forecast to t1
Destination	Full route duration	t2 = departure + totalDuration	Nearest forecast to t2

Table 3. Route-point and forecast-time matching concept

4.2 Detailed Algorithm Descriptions

The third and most important algorithm is route–weather synchronization and timeline generation. The route engine provides travel duration in seconds, but the weather API provides multiple forecast records at discrete timestamps. The application bridges these two data models by constructing three target times: the departure time itself, the midpoint time computed as departure plus half the route duration, and the destination time computed as departure plus the full route duration. Those times are then used as anchors against which forecast timestamps are compared. In effect, the application treats weather matching as a nearest-neighbour problem in one-dimensional time space.

The helper function `getClosestForecast()` implements this nearest-time selection. It first converts the target timestamp from milliseconds to seconds to match the units returned by the OpenWeather `dt` field. It then reduces the forecast list by repeatedly comparing the absolute difference between the target time and each candidate timestamp. The record with the smallest absolute difference is returned as the best available forecast for that route point. This method is computationally simple and appropriate for the moderate list size returned by a 5-day 3-hour forecast API.

A fourth algorithm governs warning generation. After forecast records have been matched, the application loops through the selected weather objects and extracts the condition code, main

condition label, and wind speed. If the code falls in the 2xx range, a thunderstorm warning is added. If the code falls in the 5xx range or the main condition text equals 'Rain', a rain warning is added. If the code falls in the 7xx range, a reduced-visibility warning is added. If wind speed exceeds 10 metres per second, a strong-wind warning is added. Warnings are stored in a JavaScript `Set`, which ensures that duplicate messages are not repeated across route points.

A fifth and final algorithm concerns frontend orchestration. The main planning button click listener performs validation, sets the departure time, shows loading states, clears previous warnings and weather cards, resolves coordinates, and triggers route calculation. The `routesfound` event then updates the summary, adds a midpoint marker, computes times, and calls the weather workflow. Once weather retrieval and matching finish, individual card containers are updated through `renderWeather()`, while warning banners are appended into the warnings section. The button state is then restored. This is a strong example of event-driven single-page application logic without any frontend framework.

For clarity, the technical workflow can be summarized as follows. Input text is converted to coordinates. Coordinates are converted to a route. The route is converted to route points and route duration. Route duration is converted to target times. Target times are compared with forecast timestamps. Matched forecasts are converted into visual cards and warnings. This sequence is the backbone of the application and should be highlighted in any technical evaluation of the project.

Road Mapping and Path Computation Algorithm

- Read source and destination text from the UI.
- Resolve the inputs into latitude–longitude coordinates using stored autocomplete values or fallback geocoding.
- If either point cannot be resolved, stop the workflow and display an error message.
- Request a drivable route between the two coordinates using Leaflet Routing Machine.
- Receive the route polyline, total distance, and total travel duration.
- Compute the midpoint index as the central element of the route coordinate array and extract the midpoint coordinate.

- Display the route on the map and update the route summary section.

Weather Forecast Extraction and API Integration Algorithm

- Prepare a list of coordinates corresponding to start, midpoint, and destination.
- For each route point, construct an OpenWeather forecast request URL with latitude, longitude, API key, and units.
- Fetch all forecast responses concurrently using asynchronous JavaScript promises.
- Validate each response and extract the forecast list from the JSON object.
- Store the raw forecast arrays for later time-based selection.

Route–Weather Synchronisation and Timeline Generation Algorithm

- Read the user-selected departure time; if absent, use the current time.
- Use the total route duration returned by the routing engine to compute target timestamps for start, midpoint, and destination.
- Convert each target time into the same unit system used by the forecast API.
- For each route point, compare all forecast timestamps with the target time.
- Select the forecast record having the smallest absolute time difference.
- Return the matched start, midpoint, and destination forecast records.

Warning Generation Algorithm

- Inspect the selected weather record at each route point.
- Check for thunderstorm condition codes in the 2xx range.
- Check for rain condition codes in the 5xx range or a main condition label of Rain.
- Check for atmospheric visibility-related codes in the 7xx range.
- Check whether wind speed exceeds the chosen threshold of 10 m/s.
- Add appropriate human-readable warning messages into a unique collection.
- Render those warnings as styled alert cards in the interface.

Frontend Orchestration Algorithm

- Capture user input events and perform debounced autocomplete search.
- On action button click, validate input fields and set loading state.

- Clear previous route summary, warning, and weather outputs.
- Trigger route calculation; on success, compute midpoint and timeline.
- Trigger forecast retrieval and nearest-time selection.
- Render route summary, weather cards, and warnings; then restore button state.

CHAPTER 5

DATASET AND TECHNOLOGY STACK

5.1 Data Sources Used by the System

The proposed application does not rely on a single static CSV file or locally stored relational database. Instead, the term 'dataset' in this project refers to operational live data sources accessed through external services. This distinction is important because the application is not trained on historical observations and does not persist route or weather history in a backend. Its outputs are generated dynamically for each user request from current map and forecast services.

OpenStreetMap functions as the foundational geospatial source for the system. The route planner depends on a routable road graph derived from OpenStreetMap, while Nominatim geocoding also operates over OpenStreetMap place data. This makes OpenStreetMap central not only to rendering but to the semantic interpretation of route endpoints. The project overview specifically cites OpenStreetMap and Nominatim as the services used for geocoding and reverse geocoding.

The Nominatim service is used in two ways in the current implementation. First, it powers text search and autocomplete suggestions for the start and destination fields. Second, it supports reverse geocoding of the route midpoint so that a nearby locality name can be displayed instead of an anonymous pair of coordinates. In the code, both direct geocoding and reverse-geocoding requests are built as `fetch()` calls against Nominatim endpoints.

For route generation, the application uses Leaflet Routing Machine as the routing control in the browser. This library acts as the route-request and route-render orchestration layer, allowing the application to submit origin and destination waypoints and receive route information in a way that integrates naturally with the Leaflet map. The project overview describes this route

subsystem as 'Leaflet & Leaflet Routing Machine (OSRM)'. The uploaded code confirms that route computation is performed through `L.Routing.control` and that the route summary and coordinate array are read from the returned route object.

OpenWeather serves as the meteorological data source. The project overview specifies the 5-day future forecast product returned in 3-hour chunks, and the implementation matches that description by calling `api.openweathermap.org/data/2.5/forecast`. Forecast entries contain timestamps and weather variables including temperature, humidity, wind speed, icon codes, and descriptive labels. Because the forecast is time-indexed rather than route-indexed, the application must perform the additional matching logic described in the algorithm chapter.

Data source / technology	Role in the system	Evidence from provided material
OpenStreetMap	Base road-network and map context for routing-related services	Named in project overview and used through tiles/ geocoding/routing stack
Nominatim	Autocomplete search, geocoding, and reverse geocoding	Functions `geocode()` and midpoint reverse lookup in script.js
Leaflet	Interactive full-screen map rendering	Map element `#map` and `L.map()` initialization
Leaflet Routing Machine	Route request, route drawing, and route event handling	`L.Routing.control(...)` in script.js
OSRM-backed route service	Road path, route distance, and duration	Described in project overview as route-calculation engine
OpenWeather Forecast API	5-day forecast data in 3-hour chunks	`fetchForecast()` uses OpenWeather forecast endpoint
HTML / CSS / JavaScript	Structure, styling, and client-side logic	Uploaded `index.html`, `styles.css`, and `script.js`

Table 2. External services used by the application

5.2 Technology Stack

The project also includes a clearly defined frontend technology stack. `index.html` provides the document structure and content placeholders, `styles.css` provides visual presentation and responsive layout, and `script.js` provides all behavior and data orchestration. The map is rendered with Leaflet, route control is provided by Leaflet Routing Machine, and the application uses only browser-native JavaScript rather than a component framework such as React or Vue. This architecture reduces complexity and supports the project goal of remaining easy to host and easy to understand.

Layer	Technology	Implementation role
Document structure	HTML	Defines map container, inputs, summary zone, warnings, and weather cards
Presentation	CSS	Creates glassmorphism panel, responsive layout, and alert styling
Client logic	JavaScript	Coordinates API requests, timing logic, DOM updates, and warnings
Map rendering	Leaflet	Displays the interactive full-screen map
Routing control	Leaflet Routing Machine	Requests routes and exposes route events and geometry
Route service	OSRM-backed service	Supplies route polyline, distance, and duration
Geocoding	Nominatim	Place search, autocomplete, and reverse geocoding
Weather service	OpenWeather Forecast API	Supplies time-indexed forecast records

Table 4. Technical stack and implementation role

CHAPTER 6

MODULE DESCRIPTION AND IMPLEMENTATION DETAILS

6.1 User Interface Module

The interface uses a full-screen map with an overlay panel rather than a traditional multi-page layout. This design allows the map to remain constantly visible while all interaction and result interpretation occur within a single visual frame. The title, subtitle, source and destination inputs, current-location button, departure-time picker, route summary cards, warning alerts, and weather cards are all grouped in a progressive hierarchy so that the user is not shown irrelevant sections until the route is successfully computed.

6.2 Current Location and Input Validation Module

The code supports a current-location button for the start field through the browser geolocation API. When the user presses the button, the app attempts to capture the user's current coordinates, reverse-geocode them into a readable label, and populate the source field. Input validation is also present: the main button checks that both source and destination text are available before continuing. This combination of validation and convenience improves usability.

6.3 Route Summary Module

After routing succeeds, the application formats and displays total distance in kilometers and estimated time in minutes. These values are shown together with short labels for source and destination. The route summary serves two purposes: it provides the user with standard navigation information and also acts as the basis for later route–weather interpretation because the duration value is required for timeline generation.

6.4 Weather Card Module

Each weather card contains a location label, an arrival-time label, and a content region into which weather details are injected. The render function populates the card with forecast time, icon, rounded temperature, textual description, wind speed, and humidity. This card-based presentation is compact enough for a mini-project interface while still exposing the forecast details needed for travel judgement.

6.5 Warning Module

The warning module transforms meteorological readings into direct advice. It does not simply print raw weather codes. Instead, it converts those into human-readable hazard alerts such as thunderstorm conditions, slippery roads due to rain, reduced visibility, or strong wind. This is important from a human–computer interaction perspective because users travel with decisions, not with numeric codes.

6.6 Event-Driven Control Flow

The app operates through event listeners rather than sequential server-side request chains. Input events drive autocomplete. Button clicks trigger route and weather workflows. Routing completion triggers midpoint, timeline, and weather orchestration. This event-driven model is natural for browser applications and is reflected clearly in the code structure.

CHAPTER 7

OUTPUT, DISCUSSION, ADVANTAGES AND LIMITATIONS

7.1 Output and Discussion

The key output of the application is a synchronized route-weather guidance view. The user receives not only a visible route, but also a route summary, a weather interpretation for three route points, and warnings relevant to those points. This makes the application more informative than a simple route viewer or city forecast viewer.

The uploaded paper is especially useful for interpreting the significance of this output. Ye et al. argue that navigation systems gain value when route information is fused with weather information, thereby enabling more informed route selection. Our application supports that idea, but in a focused form: instead of displaying a line graph of route temperature or multiple route alternatives, it gives a concise forecast snapshot at key route milestones and converts those into direct warnings.

The paper also discusses a limitation of fixed-time temperature representation and notes the importance of more dynamic updates. That observation is relevant here as well. Our app provides forecast guidance for the moment of planning, but does not continuously update the route or the forecast as the traveller progresses. It is therefore a predictive planning tool rather than a live route-tracking system.

7.2 Advantages

- Frontend-only deployment keeps hosting simple and reduces infrastructure complexity.
- Open and free-tier services make the system suitable for an academic mini-project.
- Route-specific forecast matching provides richer decision support than destination-only weather display.
- Warning banners translate raw weather conditions into immediate travel guidance.
- The interface is visually cohesive and readable, combining spatial and card-based presentation.

- The implementation is educationally valuable because it demonstrates practical integration of multiple APIs in a browser environment.

7.3 Limitations

- Nominatim is effective for towns and cities but less reliable than commercial services for highly specific places and local businesses.
- The OpenWeather 5-day product returns forecasts in 3-hour blocks, so the selected forecast is only an approximation of exact arrival-time weather.
- Free APIs can impose rate limits, which may affect scalability under heavy simultaneous use.
- The midpoint is derived from the route coordinate array and may not exactly represent the temporal halfway point of a journey on a highly curved route.
- The system currently works with a single route and three major route points rather than multiple waypoints or alternative route comparisons.
- The application does not continuously recompute its guidance in response to live movement, changing traffic, or newly updated forecasts.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The present report demonstrates that the final web app can be understood as a coherent, technically grounded response to a recognized gap in navigation systems: the absence of route-aware weather guidance. By combining geocoding, route computation, midpoint labeling, route timeline creation, forecast retrieval, nearest-time selection, and hazard generation, the application transforms separate route and weather services into a single travel-planning workflow.

The uploaded paper by Ye et al. strengthens the report by providing literature support for the broader concept of weather-aware route decision support. At the same time, the essence of the present system remains distinct and has been preserved throughout this revised report. The web app we created is a frontend-only route-weather planner focused on start, midpoint, and destination forecast alignment and warning generation. It is not a full commercial navigation engine and does not claim to be one. Its value lies in demonstrating the practical integration of open geospatial and forecast services in a clear, educational, and user-facing design.

8.2 Future Scope

- Increase route-point density by sampling multiple intermediate coordinates instead of only a single midpoint.
- Add waypoint support so that user-defined stops can be included in forecast matching.
- Support alternative route comparison and compare associated weather risks.
- Introduce a backend layer for API-key protection, response caching, and more robust orchestration.
- Use live GPS tracking to update user position and refresh route-related forecast guidance during the journey.
- Expand weather interpretation beyond rain and wind to include visibility, precipitation intensity, pressure change, or comfort metrics.
- Explore more dynamic route adaptation models inspired by the literature on static versus dynamic route-finding.

CHAPTER 9

REFERENCES

1. [1] A. Anwar, “Google Maps weather along route: Weather-aware planner,” Afi Labs Blog, Jun. 2, 2025. [Online]. Available: https://blog.afi.io/blog/google-maps-weather-along-route-weather-aware-planner/. [Accessed: Aug. 22, 2026].

2. [2] E. B. Setiawan, G. P. Husein, and A. Setiyadi, “Travel Route Recommendation System Based on Weather Prediction and Geolocation Technology,” in 2023 International Conference on Informatics Engineering, Science & Technology (INCITEST), 2023, doi: 10.1109/INCITEST59455.2023.10395920.

3. [3] X. Ye, S. Li, S. Das, and J. Du, “Enhancing routes selection with real-time weather data integration in spatial decision support systems,” Spatial Information Research, Nov. 2023, doi: 10.1007/s41324-023-00564-8.

CHAPTER 10

PUBLICATIONS